Architecture Workflow - Cover and Legend

Purpose
This architecture pack explains how departments, services, and data flows connect from recruiter input to ranked candidate outputs and CSV reporting.

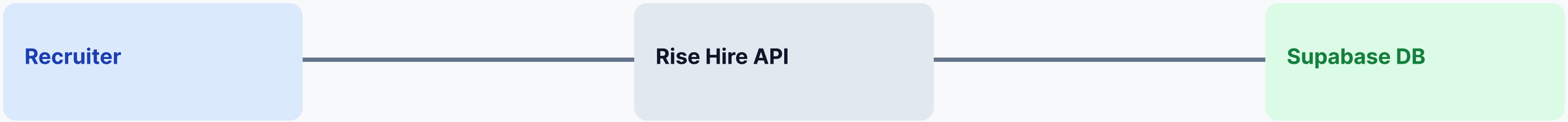
Legend
Actor = rounded box (blue)
Service = rounded box (cyan)
Data Store = dark outlined box (green label)
Async Process = dashed connector
Output = orange-labeled destination

Reading order: left to right, then top to bottom.

Department Connection Map

Connection logic: requirements → model logic → implementation → consumable outputs

System Context Diagram



Context Narrative
External actor (recruiter/front office) interacts via frontend and API.
Core platform orchestrates authentication, offer management, CV ingestion, session scoring, result ranking, and export.
Persistent storage captures offers, CV metadata, score breakdowns, and session status.
Outputs return to RH decision flow and reporting.

C4 context level: one system, clear external boundaries, explicit data store.

Backend Container and Module View

Auth Module

Offers Module

CV Module

Sessions + Scoring

Supabase PostgreSQL Storage Layer

Module contracts align with docs/API.md endpoint groups.

Runtime Sequence (Recruiter Journey)

Runtime Sequence
1) Recruiter login
2) Offer extraction/creation
3) CV uploads
4) Session creation with weights
5) Async scoring trigger
6) Poll status
7) Retrieve ranked results
8) Export CSV

Async branch: scoring runs in background while UI keeps user informed via polling.

Failure branch: failed status → show actionable error and retry path.

Scoring and Extraction Pipeline

Pipeline Flow
CV PDF → text extraction → normalized candidate profile
JD text → requirement extraction
profile + requirements → weighted scoring engine
critical skill penalty adjustment → final score + threshold
ranked candidates → results API + CSV export

Score outputs include explainable breakdown fields for recruiter confidence.

Threshold logic: green / orange / red for fast decision support.

Error Paths and Reliability Controls

Error Paths
- Auth errors (401)
- Validation errors (400/422)
- Resource errors (404)
- Scoring conflict (409)
- Upload constraints (413)
- Internal failures (500)

Reliability Controls
- Deterministic error shape: {detail}
- Async status polling and conflict handling
- Extraction confidence flags for manual review
- Explicit retry and fallback UX patterns

Design objective: failures must be visible, understandable, and recoverable.

Deployment and Integration Topology

Deployment Topology
Frontend (WordPress/Custom UI) ↔ Rise Hire FastAPI backend ↔ Supabase PostgreSQL

Integration Topology
- Auth token lifecycle on client
- Endpoint contracts enforced via docs/API.md
- Session scoring and result retrieval loop
- CSV export route for reporting

Final architecture gate: all modules connected with testable handoff contracts.